Vendredi 16/10 — ESLint, conditions et boucles

Kevin Bardoux

vendredi 16 octobre 2026

Table des matières

- Objectifs 2

Veille — Installer un paquet npm : quels risques ? (30 min) 2

- Avant la veille : `npm install` (2 min) 2
- Pourquoi cette question ? 3
- En groupe (15 min) 3
- Mise en commun et trace écrite (15 min) 4

Matin — Conditions & boucles — exercices guidés · ESLint dès le départ 4

- Rappel (5 min) 4
- Apport 1 — ESLint (5 min) 4
- ESLint — attention et vérification 6
- Exercice — `lint-moi.js` (20 min) 6
- Apport 2 — `if`, `else if`, `else` (15 min) 6
- Conditions — attention et vérification 8
- Exercice 06 — Majeur ou mineur ? (20 min) 8
- Apport 3 — La boucle `while` (8 min) 8
- `while` — attention et vérification 9
- Apport 4 — La boucle `for` (7 min) 9
- `for` — attention et vérification 10
- À retenir — ce matin 11
  - Si le temps le permet — Le reste `%`, puis 07 et 08 11
  - Pour aller plus loin 12
- Débrief 12

Après-midi — Groupe B : projet & consolidation 13

Au programme 13
Sources 13

> Note
>
> **Compétences visées** : résolution de problème · apprendre en continu **Livrable** : `lint-moi.js` corrigé ; exercice 06 au vert ; `npx eslint .` sans alerte sur vos fichiers.

Objectifs

- Citer deux risques liés à l'installation de paquets npm.
- Corriger les alertes d'ESLint.
- Écrire une condition avec `if`, `else if` et `else`, en traitant d'abord les entrées invalides.
- Écrire une boucle `while` et une boucle `for`.

Comment se passe la matinée. Lancez `npm install` dès votre arrivée (étape 1 ci-dessous) : il télécharge ESLint pendant la veille. Chaque notion suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification. Un encadré « À retenir » termine la matinée.

Veille — Installer un paquet npm : quels risques ? (30 min)

Avant la veille : `npm install` (2 min)

1. Terminal : `cd ~/dwwm/semaine-02`, puis `npm install`.
2. Laissez-le travailler pendant la veille.
3. **Réussi si** : un dossier `node_modules` apparaît.

`npm install` lit le fichier `package.json` du dossier et télécharge les paquets qu'il cite : ici ESLint, l'outil de ce matin. Il demande internet.

Vous devez voir, à la fin (les nombres et la durée peuvent varier) :

```
PS C:\Users\stagiaire\dwwm\semaine-02> npm install

added 79 packages, and audited 80 packages in 10s
```

```
18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

La ligne `packages are looking for funding` n'est pas une erreur : certains auteurs de paquets acceptent des dons. `found 0 vulnerabilities` : npm ne connaît aucune faille dans ces versions.

Pourquoi cette question ?

- `npm install` télécharge ESLint… et bien d'autres paquets.
- Chacun a été écrit par des inconnus.
- Question du jour : quels risques, et quelles précautions ?

Pour vous faire une idée, ouvrez le dossier `node_modules` que `npm install` vient de créer dans `dwwm/semaine-02` : combien de dossiers contient-il, alors que `package.json` ne cite que trois paquets ?

En groupe (15 min)

1. **Groupe 1** : la page d'npm sur les menaces (s. d.).
2. Groupe 1 : qu'est-ce que le *typosquatting* ? Que fait npm contre lui ?
3. **Groupe 2** : le risque A03 de l'OWASP Top 10 (2025).
4. Groupe 2 : relevez deux précautions recommandées.
5. Dans chaque groupe : un lecteur, un secrétaire, un porte-parole.

Les deux pages sont en anglais : *npm: Threats and Mitigations* (npm, s. d.) et *A03 Software Supply Chain Failures* (OWASP Foundation 2025). Ne traduisez pas tout : cherchez les titres de sections et les listes. Le **lecteur** lit la page à voix haute ou la parcourt pour le groupe ; le **secrétaire** note les réponses ; le **porte-parole** les présente à la mise en commun. Si vous préférez ne pas prendre la parole devant le groupe, prenez le rôle de lecteur ou de secrétaire.

Mise en commun et trace écrite (15 min)

1. Chaque porte-parole présente ce que son groupe a trouvé (3 min).
2. Seul : notez une précaution à prendre avant un `npm install`.

> **i** Note
>
> Gardez cette feuille avec celle de la semaine 1 : votre journal de veille commence en semaine 4.

Matin — Conditions & boucles — exercices guidés · ESLint dès le départ

Rappel (5 min)

1. Quelle commande lance les tests d'un exercice ?
2. Que montre la ligne `-` d'un test rouge ?
3. Un script Scratch sans événement : que se passe-t-il ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — ESLint (5 min)

- **Pourquoi** : trouver des erreurs **avant** de lancer le programme.
- **ESLint** relit votre code et signale chaque écart à une règle.

Pourquoi. Une faute de frappe dans un nom de variable ne se voit qu'au moment où le programme plante, parfois longtemps après. ESLint la trouve en relisant le code, sans le lancer.

Définition. ESLint est un outil qui relit votre code sans l'exécuter et signale chaque ligne qui enfreint une règle (ESLint, s. d.-b). Chaque alerte donne la ligne et la colonne, un message, puis le **nom de la règle**. Les règles du cours : `===` obligatoire (`eqeqeq`) (ESLint, s. d.-a), `const` quand la valeur ne change pas (`prefer-const`) (ESLint, s. d.-e), jamais `var` (`no-var`) (ESLint, s. d.-d). Une variable déclarée mais jamais utilisée est signalée par `no-unused-vars` (ESLint, s. d.-c). Le dossier contient déjà un `package.json` et un `eslint.config.js` prêts.

Exemple. `npx eslint lint-moi.js` sur le fichier de départ. ESLint écrit d'abord le chemin du fichier, puis une ligne par alerte : ligne:colonne, gravité (`error`), message, nom de la règle.

```
PS C:\Users\stagiaire\dwwm\semaine-02> npx eslint lint-
moi.js

C:\Users\stagiaire\dwwm\semaine-02\lint-moi.js
   8:1   error  Unexpected var, use let or const
instead             no-var
   9:5   error  'greeting' is never reassigned. Use
'const' instead  prefer-const
  10:7   error  'unused' is assigned a value but never
used          no-unused-vars
  12:15  error  Expected '===' and instead saw '=='
eqeqeq
  17:13  error  'mesage' is not defined
no-undef

✖ 5 problems (5 errors, 0 warnings)
  2 errors and 0 warnings potentially fixable with the
`--fix` option.
```

La dernière ligne propose l'option `--fix`, qui corrige seule certaines alertes : ne l'utilisez pas cette semaine, corrigez à la main pour comprendre chaque règle.

Pas à pas. Le formateur fait chaque étape sur la première alerte ; vous la refaites aussitôt.

1. Tapez `npx eslint lint-moi.js`, puis Entrée. Cinq alertes s'affichent.
2. Lisez la première : ligne `8`, colonne `1`, message `Unexpected var…`, règle `no-var`.
3. Dans VS Code, ouvrez `lint-moi.js` et allez à la ligne 8.
4. Remplacez `var` par `const`. Enregistrez.
5. Relancez `npx eslint lint-moi.js` : il reste quatre alertes.

La fiche Lire un message d'erreur reprend la lecture d'une alerte d'ESLint.

ESLint — attention et vérification

> ⚠ Attention
>
> Corrigez une alerte à la fois, en partant du haut, et relancez ESLint après chacune : une correction change parfois les alertes suivantes.

- **Vérifiez** : dans `12:15`, que désignent `12` et `15` ?

Exercice — `lint-moi.js` (20 min)

1. Ensemble (5 min) : la première alerte, comme dans le pas à pas.
2. Seuls (10 min) : corrigez les quatre autres, une par une.
3. `node --test lint-moi.test.js` : le test doit passer.
4. `npx eslint .` : aucune alerte dans tout le dossier.
5. **Réussi si** : `npx eslint .` n'affiche rien.

`npx eslint .` vérifie tous les fichiers du dossier, y compris les exercices que vous n'avez pas encore faits : leurs fichiers de départ sont écrits pour passer ESLint. À partir d'aujourd'hui, un exercice est **fini** quand ses tests passent **et** qu'ESLint n'affiche aucune alerte.

Apport 2 — `if`, `else if`, `else` (15 min)

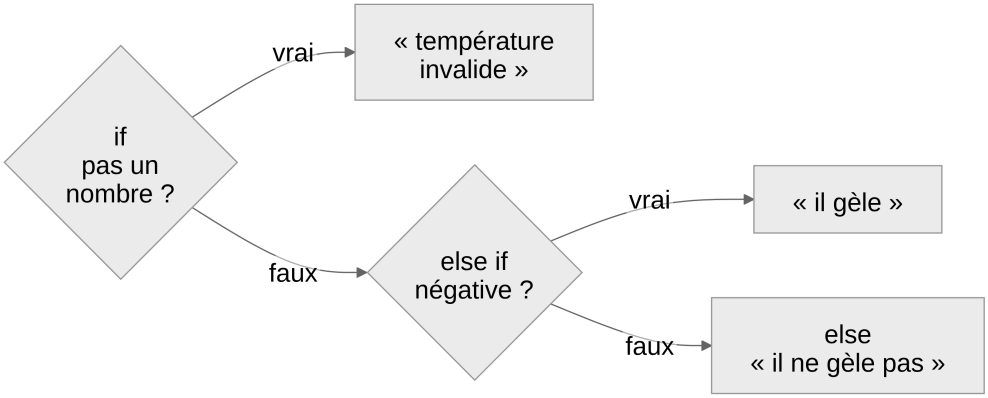
- **Pourquoi** : agir différemment selon la valeur reçue.
- **Condition** : la première branche vraie s'exécute, et elle seule.

Pourquoi. Un programme ne fait pas toujours la même chose : « majeur » ou « mineur » selon l'âge, un message d'erreur si l'âge n'est pas un nombre.

Définition. `if` exécute un bloc si une condition est vraie ; `else if` teste une autre condition ; `else` précise quoi faire sinon (MDN Web Docs, s. d.-b). Les conditions sont testées dans l'ordre : la première vraie gagne, les suivantes sont ignorées.

Pour écrire une condition : comparer avec `<`, `<=`, `>`, `>=`, `===`, `!==` ; combiner avec `&&` (et), `||` (ou), `!` (non). `Number('abc')` donne `NaN`

(*Not a Number*) : `Number.isNaN()` le détecte (MDN Web Docs, s. d.-c). **Style défensif** : traitez d'abord les entrées invalides.



Les conditions sont testées dans l'ordre (ici sur temperature) : la première vraie gagne, et seule sa branche s'exécute ; else, si aucune n'est vraie.

Exemple.

```
const temperature = Number(process.argv[2]);

if (Number.isNaN(temperature)) {
  console.log('température invalide');
} else if (temperature < 0) {
  console.log('il gèle');
} else {
  console.log('il ne gèle pas');
}
```

Pas à pas. Le formateur crée le fichier `meteo.js` dans VS Code avec le code ci-dessus ; vous le recopiez, puis :

1. Lancez `node meteo.js abc`. Le programme affiche `température invalide` : la première condition est vraie.
2. Lancez `node meteo.js -5`. Il affiche `il gèle` : la première est fausse, la deuxième vraie.
3. Lancez `node meteo.js 12`. Il affiche `il ne gèle pas` : aucune condition n'est vraie, `else` s'exécute.
4. Lancez `node meteo.js 0`. Il affiche `il ne gèle pas` : `0 < 0` est faux.

Conditions — attention et vérification

> ⚠ Attention
>
> `Number('')` donne `0`, pas `NaN` : un texte vide passe pour zéro. Et `=` dans une condition ne compare pas : écrivez `===`.

- **Vérifiez** : qu'affiche `node meteo.js 0` ? Prédisez, puis lancez.

Exercice 06 — Majeur ou mineur ? (20 min)

1. Ensemble (5 min) : écrivez la première condition, l'âge invalide.
2. Seuls : `majeur` à partir de 18 ans, `mineur` avant.
3. `node --test 06-majorite.test.js`, puis `npx eslint .`.
4. **Réussi si** : tests au vert, aucune alerte.

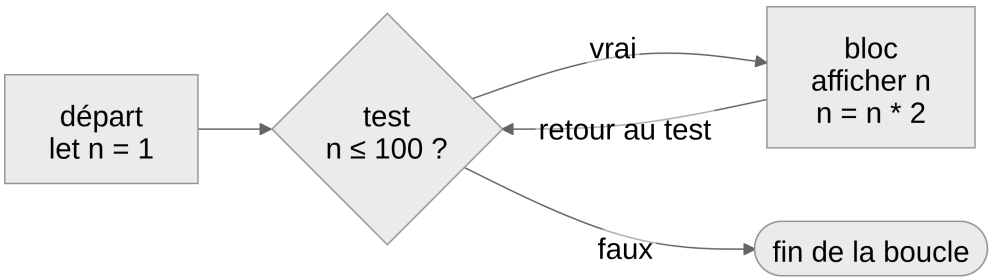
- `node 06-majorite.js 18` affiche `majeur`.
- `node 06-majorite.js 17` affiche `mineur`.
- Un texte ou un âge négatif : `âge invalide`.
- Pensez au cas limite : 18 ans, comme en semaine 1.

Apport 3 — La boucle `while` (8 min)

- **Pourquoi** : répéter une action sans recopier la ligne.
- **`while`** : tant que la condition est vraie, on recommence.

Pourquoi. Afficher 3, 2, 1 se fait en trois lignes ; afficher de 100 à 1 en demanderait cent. Une boucle répète la même action autant de fois que nécessaire.

Définition. `while (condition) { … }` répète le bloc tant que la condition est vraie (MDN Web Docs, s. d.-a). Quelque chose doit changer dans le bloc, sinon la condition reste vraie pour toujours.



Tant que la condition est vraie, le bloc s'exécute, puis on revient au test ; quand elle devient fausse, la boucle s'arrête. (≤ s'écrit <= en JavaScript.)

Exemple.

```
let n = 1;
while (n <= 100) {
  console.log(n);
  n = n * 2;
}
```

Pas à pas. Le formateur suit la boucle au tableau, tour par tour ; vous remplissez le tableau sur votre feuille.

1. Avant la boucle, `n` vaut `1`. `1 <= 100` est vrai : on affiche `1`, puis `n` vaut `2`.
2. `2 <= 100` est vrai : on affiche `2`, puis `n` vaut `4`.
3. Continuez : `4`, `8`, `16`, `32`, `64`.
4. `n` vaut `128` : `128 <= 100` est faux. La boucle s'arrête.
5. Vérifiez en lançant le programme : il affiche 1, 2, 4, 8, 16, 32, 64.

`while` — attention et vérification

> ⚠ Attention
>
> Si `n` ne change jamais, la boucle ne s'arrête pas : le terminal se fige. Arrêtez-la avec `Ctrl+C` (windows), `Ctrl+C` (mac), puis ajoutez la ligne qui change `n`.

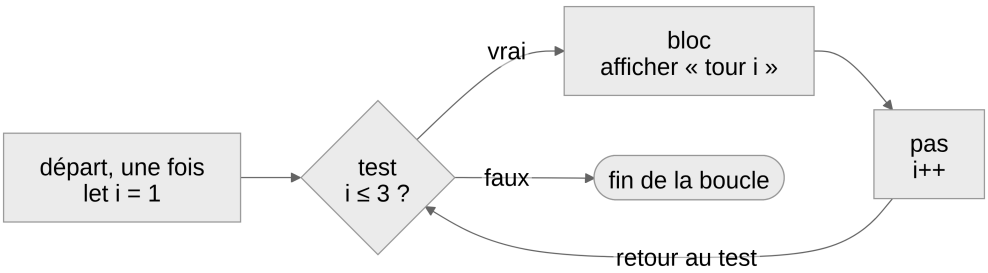
- **Vérifiez** : avec `n = n * 3`, combien de nombres s'affichent ?

Apport 4 — La boucle `for` (7 min)

- **Pourquoi** : une boucle qui compte, écrite sur une seule ligne.
- **`for`** : départ, condition et pas, réunis entre parenthèses.

Pourquoi. La plupart des boucles comptent : de 1 à 3, de 10 à 0. Avec `while`, le départ, la condition et le pas sont sur trois lignes différentes, et on oublie facilement le pas.

Définition. `for (départ; condition; pas) { … }` exécute le départ une fois, puis répète le bloc tant que la condition est vraie, en appliquant le pas après chaque tour (MDN Web Docs, s. d.-a). `i++` ajoute 1 à `i` (comme `i = i + 1`) ; `i--` retire 1.



La boucle for réunit le départ, le test et le pas : départ une fois, puis test, bloc, pas, et retour au test. (≤ s'écrit <= en JavaScript.)

Exemple.

```
for (let i = 1; i <= 3; i++) {
  console.log(`tour ${i}`);
}
```

Pas à pas. Le formateur suit la boucle au tableau ; vous remplissez le tableau sur votre feuille.

1. Départ : `let i = 1`, une seule fois.
2. Condition : `1 <= 3` est vrai : on affiche `tour 1`.
3. Pas : `i++`, donc `i` vaut `2`. `2 <= 3` : on affiche `tour 2`.
4. Pas : `i` vaut `3`. `3 <= 3` : on affiche `tour 3`.
5. Pas : `i` vaut `4`. `4 <= 3` est faux : la boucle s'arrête.

`for` — attention et vérification

> ⚠ Attention
>
> Un test qui affiche `ETIMEDOUT` : votre programme a dépassé le temps permis. C'est presque toujours une boucle infinie : vérifiez le pas.

- **Vérifiez** : écrivez une boucle `for` qui affiche 3, 2, 1.

À retenir — ce matin

> À retenir
>
> - **ESLint** : `npx eslint .` ; une alerte à la fois, du haut vers le bas.
> - **Conditions** : l'entrée invalide d'abord ; la première vraie gagne.
> - **`while`** : quelque chose doit changer, sinon la boucle ne s'arrête pas.
> - **`for`** : départ, condition, pas, sur une ligne.

Le détail, notion par notion :

- **ESLint** : il relit le code sans le lancer ; chaque alerte donne ligne, colonne, message et règle ; un exercice est fini quand les tests passent et qu'ESLint n'affiche rien.
- **Conditions** : `if`, `else if`, `else` ; la première condition vraie gagne ; traitez d'abord les entrées invalides (`Number.isNaN`).
- **`while`** : répète tant que la condition est vraie ; quelque chose doit changer dans le bloc ; `Ctrl+C` (windows), `Ctrl+C` (mac) arrête une boucle infinie.
- **`for`** : départ, condition et pas sur une ligne ; `i++` ajoute 1.

Si le temps le permet — Le reste `%`, puis 07 et 08

L'opérateur reste. `a % b` donne le **reste** de la division de `a` par `b` : `12 % 2` vaut `0`, `7 % 2` vaut `1`. Un nombre est pair si son reste par 2 vaut `0`. Le résultat de `%` a le signe du premier nombre (MDN Web Docs, s. d.-d) : `-3 % 2` vaut `-1`, pas `1`. Testez donc « reste égal à 0 », jamais « reste égal à 1 ».

Exercice 07 — Pair ou impair ?

1. `node 07-pair-impair.js 12` affiche `12 est pair`.
2. `node 07-pair-impair.js 7` affiche `7 est impair`.
3. Un texte : `nombre invalide`. Les tests essaient aussi des nombres négatifs.
4. `node --test 07-pair-impair.test.js`, puis `npx eslint .`.

Exercice 08 — Compte à rebours, `while` puis `for`.

1. `node 08-compte-a-rebours.js 3` affiche 3, 2, 1, puis `Décollage !`. Un texte ou un nombre négatif : `nombre invalide`.
2. Étape 1 : écrivez-le avec une boucle `while`. Tous les tests passent sauf le dernier (« utilise une boucle for ») : c'est normal.
3. Gardez une copie de votre version `while` (dans un commentaire ou un autre fichier).
4. Étape 2 : réécrivez la boucle avec `for`. Le dernier test passe.

> Indice — étape 2
>
> Les trois parties de votre boucle `while` sont déjà là : le départ (`let n = start`), la condition (`n > 0`) et le pas (`n = n - 1`). Une boucle `for` les réunit sur une seule ligne.

Pour aller plus loin

- **09-somme** : `La somme de 1 à 4 vaut 10`, avec une boucle `for`.
- **10-fizzbuzz** : le défi de la semaine 1, en JavaScript. Fizz, Buzz, FizzBuzz : dans quel ordre écrire les conditions ?

> Indice — 09-somme
>
> Créez `let sum = 0;` avant la boucle, puis ajoutez `i` à `sum` à chaque tour. C'est « La somme » de la semaine 1, traduite du pseudo-code en JavaScript.

10-fizzbuzz s'inspire du jeu *Fizz buzz* (Wikipedia, s. d.). Les stagiaires qui veulent aller encore plus loin trouveront `11-triangle.js` (boucles imbriquées) dans le niveau 3 du parcours de remédiation.

Débrief

- Pourquoi traiter les entrées invalides en premier ?
- Quand choisir `while`, quand choisir `for` ?
- Quelle alerte ESLint avez-vous le plus rencontrée ?

Après-midi — Groupe B : projet & consolidation

> Note
>
> **Groupe B** en formation de 12 h 45 à 17 h ; le groupe A est au sport (13 h 15 – 16 h). Pas de notion nouvelle : on termine, on consolide, on approfondit.

Au programme

1. **Terminer** (1 h) : `lint-moi` et 06, tests et ESLint.
2. **Consolider** (1 h) : 07, puis 08 (`while`, puis `for`).
3. **Papier** (45 min) : 06 et 08 sur une feuille, puis à la machine.
4. **Relire** (16 h 15) : un exercice d'un camarade, en binôme.

Aucune notion nouvelle : ce sont ESLint, les conditions et les boucles de ce matin. 07 et 08 sont décrits dans « Si le temps le permet », plus haut. Refaire 06 et 08 sur papier prépare l'ECF du 23/10, qui se fait sans machine pour sa première partie. Pour la relecture : l'un lit son code à voix haute, l'autre suit et note une remarque, puis vous échangez les rôles. Le groupe A fera le même entraînement sur papier avec la fiche de révision du mercredi 21/10.

Critères de réussite de l'après-midi

- ☐ `lint-moi`, 06, 07 et 08 passent les tests et ESLint
- ☐ 06 et 08 sont écrits sur papier, sans regarder

Sources

ESLint. s. d.-a. « eqeqeq ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/rules/eqeqeq.

ESLint. s. d.-b. « Getting Started with ESLint ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/use/getting-started.

ESLint. s. d.-c. « no-unused-vars ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/rules/no-unused-vars.

ESLint. s. d.-d. « no-var ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/rules/no-var.

ESLint. s. d.-e. « prefer-const ». Consulté le 26 septembre 2026. https://eslint.org/docs/latest/rules/prefer-const.

MDN Web Docs. s. d.-a. « Boucles et itérations - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Loops_and_iteration.

MDN Web Docs. s. d.-b. « Contrôle du flux d'instructions et gestion des erreurs - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Control_flow_and_error_handling.

MDN Web Docs. s. d.-c. « Number : méthode statique isNaN() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Number/isNaN.

MDN Web Docs. s. d.-d. « Reste (». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Remainder.

npm. s. d. « npm: Threats and Mitigations ». Consulté le 26 septembre 2026. https://docs.npmjs.com/threats-and-mitigations.

OWASP Foundation. 2025. « A03 Software Supply Chain Failures - OWASP Top 10:2025 ». https://top10.owasp.org/2025/A03_2025-Software_Supply_Chain_Failures.

Wikipedia. s. d. « Fizz buzz ». Consulté le 26 septembre 2026. https://en.wikipedia.org/wiki/Fizz_buzz.